PandaDaw - Documentación Técnica Completa

Versión: 1.0
Fecha: Febrero 2026
Equipo: 3 Desarrolladores
Tarifa: 25€/hora

---

1. Resumen Ejecutivo

PandaDaw es una plataforma de comercio electrónico completa desarrollada con tecnología .NET. El proyecto consiste en dos aplicaciones principales:

- **PandaBack**: API REST y servicios backend
- **PandaDawRazor**: Interfaz de usuario con Razor Pages y Blazor

Características Principales

- 🛒 Carrito de compras persistente
- 💖 Sistema de favoritos
- ⭐ Valoraciones y reseñas en tiempo real
- 👤 Sistema de autenticación y autorización
- 📦 Gestión de productos (Admin)
- 💳 Sistema de pedidos y pagos
- 🔔 Notificaciones en tiempo real
- 📱 Diseño responsivo con Tailwind CSS + DaisyUI

---

2. Arquitectura del Sistema

2.1 Diagrama de Arquitectura General

```
flowchart TB
    subgraph Client["Navegador Web"]
        UI[Tailwind CSS + DaisyUI]
        Blazor[Componentes Blazor]
        JS[JavaScript AJAX]
    end

    subgraph Frontend["PandaDawRazor"]
        Pages[Páginas Razor]
        Components[Componentes]
        Services[Servicios Client]
    end

    subgraph Backend["PandaBack"]
        API[API REST Controllers]
        ServicesC[Service Layer]
```

```
        Repositories[Repository Layer]
        DB[(SQLite Database)]
    end

    UI --> Pages
    Blazor --> Components
    JS --> Pages

    Pages --> API
    Components --> Services
    Services --> API

    API --> ServicesC
    ServicesC --> Repositories
    Repositories --> DB
```

2.2 Patrón de Diseño

```
flowchart LR
    subgraph Presentation["Capa de Presentación"]
        Pages
        Components
    end

    subgraph Business["Capa de Negocio"]
        Services
        Validators
    end

    subgraph Data["Capa de Datos"]
        Repositories
        DbContext
    end

    Presentation --> Business
    Business --> Data
```

---

3. Estructura del Proyecto

3.1 Proyectos en la Solución

```
graph TD
    Solution["PandaDaw Solution"]

    PandaBack["PandaBack (Class Library)"]
    PandaDawRazor["PandaDawRazor (Web App)"]
    Tests["Tests (Unit Tests)"]
    Playwright["PandaDaw-Playwright (E2E Tests)"]
```

```
    Solution --> PandaBack
    Solution --> PandaDawRazor
    Solution --> Tests
    Solution --> Playwright

    PandaBack --> PandaDawRazor
```

3.2 Estructura de Carpetas - PandaBack

```
PandaBack/
├── config/              # Configuraciones
│   ├── CacheConfig.cs
│   └── DotEnvLoader.cs
├── Data/               # Contexto de datos
│   ├── PandaDbContext.cs
│   └── DataSeeder.cs
├── Dtos/               # Objetos de Transferencia
│   ├── Auth/
│   ├── Carrito/
│   ├── Favoritos/
│   ├── Productos/
│   ├── Valoraciones/
│   └── Ventas/
├── Errors/             # Manejo de errores
│   └── PandaError.cs
├── Mappers/           # Mapeo de entidades
│   ├── CarritoMapper.cs
│   ├── FavoritoMapper.cs
│   ├── ProductoMapper.cs
│   ├── UserMapper.cs
│   ├── ValoracionMapper.cs
│   └── VentaMapper.cs
├── Middleware/        # Middleware personalizado
│   └── GlobalExceptionHandler.cs
├── Models/            # Entidades del dominio
│   ├── Carrito.cs
│   ├── Categoria.cs
│   ├── EstadoPedido.cs
│   ├── Favorito.cs
│   ├── LineaCarrito.cs
│   ├── LineaVenta.cs
│   ├── Producto.cs
│   ├── Role.cs
│   ├── User.cs
│   ├── Valoracion.cs
│   └── Venta.cs
├── Repositories/      # Capa de acceso a datos
│   ├── Auth/
│   ├── Carrito/
│   ├── Favoritos/
```

```
│   ├── Productos/
│   ├── Valoraciones/
│   └── Ventas/
├── RestController/    # Controladores REST API
│   ├── AuthController.cs
│   ├── CarritoController.cs
│   ├── FavoritosController.cs
│   ├── ProductosController.cs
│   ├── ValoracionesController.cs
│   └── VentasController.cs
├── Services/         # Capa de servicios
│   ├── Auth/
│   ├── Carrito/
│   ├── Email/
│   ├── Factura/
│   ├── Favoritos/
│   ├── Productos/
│   ├── Stripe/
│   ├── Valoraciones/
│   └── Ventas/
├── Validators/       # Validadores
│   ├── Carrito/
│   ├── Favoritos/
│   ├── Productos/
│   ├── Valoraciones/
│   └── Ventas/
└── Program.cs        # Punto de entrada
```

3.3 Estructura de Carpetas - PandaDawRazor

```
PandaDawRazor/
├── Components/           # Componentes Blazor
│   ├── ContadorCarrito.razor
│   ├── NotificacionesToast.razor
│   └── ValoracionesRealtime.razor
├── Filters/             # Filtros de páginas
│   └── NavBadgePageFilter.cs
├── Pages/               # Páginas Razor
│   ├── AdminPanel.cshtml
│   ├── Api.cshtml
│   ├── Carrito.cshtml
│   ├── Detalle.cshtml
│   ├── Favoritos.cshtml
│   ├── Index.cshtml
│   ├── Login.cshtml
│   ├── Pago.cshtml
│   ├── PagoExitoso.cshtml
│   ├── Pedidos.cshtml
│   ├── Register.cshtml
│   └── Shared/
├── Services/            # Servicios cliente
```

```
│   └── NotificacionService.cs
├── wwwroot/             # Archivos estáticos
│   ├── css/
│   ├── js/
│   └── lib/
└── Program.cs
```

---

4. Modelo de Datos

4.1 Diagrama de Entidades

```
erDiagram
    USER ||--o{ CARRITO : tiene
    USER ||--o{ FAVORITO : tiene
    USER ||--o{ VALORACION : escribe
    USER ||--o{ VENTA : realiza

    PRODUCTO ||--o{ LINEA_CARRITO : contiene
    PRODUCTO ||--o{ LINEA_VENTA : contiene
    PRODUCTO ||--o{ FAVORITO : es_favorito
    PRODUCTO ||--o{ VALORACION : tiene

    CARRITO ||--o{ LINEA_CARRITO : contiene
    VENTA ||--o{ LINEA_VENTA : contiene

    USER {
        string Id PK
        string Email
        string PasswordHash
        string UserName
        string UserRole
        DateTime FechaRegistro
        bool IsDeleted
    }

    PRODUCTO {
        long Id PK
        string Nombre
        string Descripcion
        decimal Precio
        int Stock
        string Category
        string Imagen
        DateTime FechaAlta
        bool IsDeleted
    }

    CARRITO {
        string Id PK
        string UserId FK
```

```
        DateTime FechaActualizacion
    }

    LINEA_CARRITO {
        long Id PK
        string CarritoId FK
        long ProductoId FK
        int Cantidad
    }

    FAVORITO {
        long Id PK
        string UserId FK
        long ProductoId FK
        DateTime FechaCreacion
    }

    VALORACION {
        long Id PK
        long ProductoId FK
        string UsuarioId FK
        int Estrellas
        string Resena
        DateTime Fecha
    }

    VENTA {
        long Id PK
        string UserId FK
        decimal Total
        string Estado
        DateTime FechaVenta
    }

    LINEA_VENTA {
        long Id PK
        long VentaId FK
        long ProductoId FK
        int Cantidad
        decimal PrecioUnitario
    }
```

4.2 Enumeraciones

```
graph TB
    subgraph Categoria
        C1[Accesorios]
        C2[Alimentacion]
        C3[Camas]
        C4[Juguetes]
        C5[Otros]
```

```
    end

    subgraph Role
        R1[Usuario]
        R2[Admin]
    end

    subgraph EstadoPedido
        E1[Pendiente]
        E2[Procesando]
        E3[Enviado]
        E4[Entregado]
        E5[Cancelado]
    end
```

---

5. Servicios y API

5.1 Servicios Backend

```
classDiagram
    class IProductoService {
        <>
        +GetAllProductosAsync()
        +GetProductoByIdAsync(id)
        +GetProductosByCategoryAsync(category)
        +CreateProductoAsync(producto)
        +UpdateProductoAsync(id, producto)
        +DeleteProductoAsync(id)
    }

    class ICarritoService {
        <>
        +GetCarritoByUserIdAsync(userId)
        +AddLineaCarritoAsync(userId, productoId, cantidad)
        +UpdateLineaCantidadAsync(userId, productoId, cantidad)
        +RemoveLineaCarritoAsync(userId, productoId)
        +VaciarCarritoAsync(userId)
    }

    class IFavoritoService {
        <>
        +GetUserFavoritosAsync(userId)
        +AddToFavoritosAsync(userId, dto)
        +RemoveFromFavoritosAsync(id, userId)
    }

    class IValoracionService {
        <>
        +GetValoracionesByProductoAsync(productoId)
        +GetValoracionesByUserAsync(userId)
```

```
        +CreateValoracionAsync(userId, dto)
        +UpdateValoracionAsync(id, userId, dto)
        +DeleteValoracionAsync(id, userId)
    }

    class IVentaService {
        <>
        +GetVentasByUserAsync(userId)
        +GetVentaByIdAsync(id)
        +CreateVentaAsync(userId, dto)
        +UpdateEstadoAsync(id, estado)
    }

    class IAuthService {
        <>
        +LoginAsync(dto)
        +RegisterAsync(dto)
        +ValidateTokenAsync(token)
    }
```

5.2 Endpoints REST API

```
flowchart TB
    subgraph Auth
        A1["POST /api/auth/login"]
        A2["POST /api/auth/register"]
    end

    subgraph Productos
        P1["GET /api/productos"]
        P2["GET /api/productos/{id}"]
        P3["GET /api/productos/categoria/{categoria}"]
        P4["POST /api/productos (Admin)"]
        P5["PUT /api/productos/{id} (Admin)"]
        P6["DELETE /api/productos/{id} (Admin)"]
    end

    subgraph Carrito
        C1["GET /api/carrito/{userId}"]
        C2["POST /api/carrito"]
        C3["PUT /api/carrito"]
        C4["DELETE /api/carrito/{userId}/{productId}"]
    end

    subgraph Favoritos
        F1["GET /api/favoritos/{userId}"]
        F2["POST /api/favoritos"]
        F3["DELETE /api/favoritos/{id}"]
    end

    subgraph Valoraciones
```

```
        V1["GET /api/valoraciones/producto/{productoId}"]
        V2["POST /api/valoraciones"]
        V3["PUT /api/valoraciones/{id}"]
        V4["DELETE /api/valoraciones/{id}"]
    end

    subgraph Ventas
        VE1["GET /api/ventas/{userId}"]
        VE2["GET /api/ventas/{id}"]
        VE3["POST /api/ventas"]
    end
```

---

6. Interfaces de Usuario

6.1 Mapa de Páginas

```
flowchart LR
    Index["Index (Inicio)"]
    Detalle["Detalle Producto"]
    Carrito["Carrito"]
    Favoritos["Favoritos"]
    Pago["Pago"]
    PagoOk["Pago Exitoso"]
    Pedidos["Mis Pedidos"]
    Login["Login"]
    Register["Registro"]
    Admin["Admin Panel"]

    Index --> Detalle
    Index --> Carrito
    Index --> Favoritos
    Index --> Login
    Index --> Register
    Index --> Admin

    Detalle --> Carrito
    Detalle --> Favoritos

    Carrito --> Pago
    Pago --> PagoOk
    Pago --> Index

    PagoOk --> Pedidos
    Pedidos --> Index

    Login --> Index
    Register --> Index

    Admin --> Index
```

6.2 Componentes Blazor

```
flowchart TB
    subgraph Componentes["Componentes en tiempo real"]
        NT["NotificacionesToast.razor"]
        VC["ValoracionesRealtime.razor"]
        CC["ContadorCarrito.razor"]
    end

    subgraph NotificacionService["NotificacionService (Singleton)"]
        EV1["OnNotificacion"]
        EV2["OnNuevaValoracion"]
        EV3["OnCarritoActualizado"]
    end

    NT --> NotificacionService
    VC --> NotificacionService
    CC --> NotificacionService

    NotificacionService -->|"Notifica a"| NT
    NotificacionService -->|"Notifica a"| VC
    NotificacionService -->|"Notifica a"| CC
```

---

7. Flujos de Usuario

7.1 Flujo de Compra

```
sequenceDiagram
    participant U as Usuario
    participant I as Index
    participant C as Carrito
    participant P as Pago
    participant S as Stripe
    participant V as VentaService

    U->>I: Navega por productos
    I->>I: Añade al carrito (AJAX)
    I->>V: Notifica actualización

    U->>I: Va al carrito
    I->>C: Muestra carrito

    U->>C: Confirma compra
    C->>P: Redirige a pago

    U->>P: Ingresa datos pago
    P->>S: Procesa pago

    S-->>P: Pago confirmado
```

```
    P->>V: Crea venta

    V-->>U: Muestra confirmación
```

7.2 Flujo de Valoración

```
sequenceDiagram
    participant U as Usuario
    participant D as Detalle
    participant VR as ValoracionesRealtime
    participant VS as ValoracionService
    participant NS as NotificacionService

    U->>D: Visualiza producto
    D->>VR: Carga component

    U->>VR: Envía valoración
    VR->>VS: Crea valoración

    VS-->>VR: Valoración creada
    VR->>NS: Notifica nueva valoración

    NS-->>VR: Actualiza lista (real-time)
    NS-->>U: Muestra notificación toast
```

---

8. Tecnologías Utilizadas

8.1 Stack Tecnológico

Capa	Tecnología	Versión
Runtime	.NET	10.0
Frontend	ASP.NET Core Razor Pages	10.0
Componentes	Blazor Server	10.0
CSS	Tailwind CSS	3.x
UI Framework	DaisyUI	4.x
Base de Datos	SQLite	-
ORM	Entity Framework Core	10.0
Testing	xUnit	-
E2E Testing	Playwright	-
Icons	Font Awesome	6.x

8.2 Paquetes NuGet Principales

```
graph LR
    Core[".NET 10.0"]

    subgraph Data["Datos"]
        EF["Entity Framework Core"]
        EFSQL["EF Core SQLite"]
        EFProxies["EF Proxies"]
    end

    subgraph Auth["Autenticación"]
        JWT["JWT Bearer"]
        Identity["ASP.NET Identity"]
    end

    subgraph Validation["Validación"]
        Fluent["FluentValidation"]
    end

    Core --> Data
    Core --> Auth
    Core --> Validation
```

---

9. Presupuesto

9.1 Estimación de Horas por Fase

Fase	Descripción	Horas	Coste (3 devs)
Fase 1: Setup	Configuración proyecto, estructura base	40h	3.000€
Fase 2: Modelos	Entidades, DTOs, Mapeadores	60h	4.500€
Fase 3: Repositorios	Acceso a datos, CRUD	80h	6.000€
Fase 4: Servicios	Lógica de negocio	100h	7.500€
Fase 5: API REST	Controladores endpoints	40h	3.000€
Fase 6: Frontend Pages	Páginas Razor principales	120h	9.000€
Fase 7: Componentes Blazor	Notificaciones, valoraciones tiempo real	60h	4.500€
Fase 8: UI/UX	Estilos, animaciones, responsive	80h	6.000€
Fase 9: Testing	Unit tests, integración	60h	4.500€
Fase 10: Documentación	README, comentarios, guías	40h	3.000€
Fase 11: Ajustes	Bug fixes, mejoras	80h	6.000€
	TOTAL	**760h**	**57.000€**

9.2 Costes Adicionales

Concepto	Coste Estimado
Dominio (anual)	15€
Hosting (VPS/Cloud)	50€/mes × 12 = 600€/año
SSL (Let's Encrypt)	Gratis
Base de datos gestionada	0€ (SQLite local)
Total Year 1	**615€**

9.3 Resumen Budget

```
pie title Distribución de Costes
    "Desarrollo (760h)" : 57
    "Hosting año 1" : 0.6
    "Dominio" : 0.015
    "Contingencia (10%)" : 5.7
```

Concepto	Importe
Desarrollo	57.000€
Infraestructura Year 1	615€
Contingencia (10%)	5.700€
TOTAL PROYECTO	**63.315€**

9.4 Coste por Desarrollador

Métrica	Valor
Horas totales	760h
Horas por desarrollador	~253h
Coste por desarrollador	19.000€

---

10. Mejores Prácticas Implementadas

10.1 Patrones de Diseño

- ☑ **Repository Pattern**: Abstracción del acceso a datos
- ☑ **Service Layer**: Lógica de negocio encapsulada
- ☑ **DTOs**: Transferencia de datos controlada
- ☑ **Dependency Injection**: Inyección de dependencias nativa
- ☑ **Singleton Pattern**: NotificacionService

10.2 Seguridad

- ☑ **JWT Authentication**: Tokens para API
- ☑ **Password Hashing**: BCrypt para contraseñas
- ☑ **Role-based Authorization**: Admin vs Usuario
- ☑ **Input Validation**: FluentValidation
- ☑ **SQL Injection Protection**: Entity Framework

10.3 Rendimiento

- ☑ **Async/Await**: Operaciones asíncronas
- ☑ **Entity Proxies**: Lazy loading
- ☑ **AJAX**: Actualizaciones sin recarga
- ☑ **Blazor SignalR**: Tiempo real eficiente

---

11. Glosario

Término	Definición
DTO	Data Transfer Object - Objeto para transferir datos entre capas
Repository	Patrón para abstraer el acceso a datos
Service	Clase que encapsula lógica de negocio
Blazor	Framework .NET para crear interfaces web
Razor Pages	Modelo de desarrollo web basado en páginas
SignalR	Biblioteca para comunicación en tiempo real
JWT	JSON Web Token - Estándar para autenticación
FluentValidation	Biblioteca para validaciones fluent

---

12. Anexos

A. Comandos Útiles

```
# Restaurar paquetes
dotnet restore

# Compilar proyecto
dotnet build

# Ejecutar tests
dotnet test

# Ejecutar aplicación
dotnet run --project PandaDawRazor
```

```
# Añadir migración
dotnet ef migrations add InitialCreate

# Actualizar base de datos
dotnet ef database update
```

B. Variables de Entorno

```
DB_CONNECTION_STRING=Data Source=panda.db
JWT_SECRET=tu_secreto_aqui
JWT_EXPIRY_HOURS=24
```

Documento generado automáticamente para PandaDaw

Febrero 2026